



NOTRE DAME UNIVERSITY BANGLADESH

Machine Learning Lab Report-07

Course Code: CSE4214

Course Title: Machine Learning Lab

Lab Task Topic: Decision Tree & Confusion Matrix

Submitted by:

Name: Istiak Alam

ID: 0692230005101005

Batch: CSE-20

Submission Date: April 19, 2026

Submitted to:

A. H. M. Saiful Islam

Chairman, Dept of CSE

Notre Dame University Bangladesh

Table of Contents

1	Objective	1
2	Dataset Description	1
3	Decision Tree Classifier	1
3.1	Tennis as Classifier	1
4	Decision tree from Dataset to predict as a Regressor	4
5	Confusion Matrix	6
5.1	f1 score for “Dog” class	7
5.2	f1 score for “not a Dog” class	7

1 Objective

The objective of this experiment is to implement the Decision Tree algorithm for classification tasks. This experiment aims to build a model that can classify data into distinct categories based on feature values by learning decision rules from the dataset. It also focuses on understanding how tree-based models split data and evaluate their classification performance.

The objective of this experiment is to apply the Decision Tree algorithm for regression tasks. The goal is to predict continuous target values by learning patterns from input features using a tree-structured model. This experiment helps in understanding how decision trees can approximate numerical relationships and handle non-linear data.

The objective of this experiment is to evaluate the performance of a classification model using a confusion matrix. It aims to analyze the model's prediction results by comparing actual and predicted class labels, and to compute performance metrics such as accuracy, precision, recall, and F1-score for better assessment of the model.

2 Dataset Description

Three different datasets are used in this lab to demonstrate the application of regression algorithms.

Dataset Description: DT.csv

The DT.csv dataset is used for implementing Decision Tree algorithms. It contains multiple attributes representing different features of the data along with a target variable used for classification or prediction. The dataset is structured to demonstrate how a Decision Tree model splits data based on feature values to make decisions. It is suitable for understanding tree-based learning, feature importance, and rule-based classification.

Dataset Description: house_data.csv

The house_data.csv dataset contains real estate information used for predicting house prices. It includes several numerical and possibly categorical features such as area, number of bedrooms, location factors, and other property-related attributes. The target variable represents the house price. This dataset is used to analyze how different factors influence property prices and to apply regression techniques for accurate prediction.

3 Decision Tree Classifier

3.1 Tennis as Classifier

```
[1]: # Importing necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, \
    ConfusionMatrixDisplay
```

```
[2]: # Load the dataset
data = pd.read_csv('DT.csv')
```

```
[3]: data
```

```
[3]:      Day  Outlook  Temp  Humidity  Wind  PlayTennis  Unnamed: 6
0    D1    sunny   hot    high    weak          No      NaN
1    D2    sunny   hot    high  strong          No      NaN
2    D3  overcast   hot    high    weak          Yes      NaN
3    D4     rain  mild    high    weak          Yes      NaN
4    D5     rain  cold   normal   weak          Yes      NaN
5    D6     rain  cold   normal  strong          No      NaN
6    D7  overcast  cold   normal  strong          Yes      NaN
7    D8    sunny  mild    high    weak          No      NaN
8    D9    sunny  cold   normal   weak          Yes      NaN
9   D10     rain  mild   normal   weak          Yes      NaN
10  D11    sunny  mild   normal  strong          Yes      NaN
11  D12  overcast  mild    high  strong          Yes      NaN
12  D13  overcast   hot   normal   weak          Yes      NaN
13  D14     rain  mild    high  strong          No      NaN
14  D15    sunny   hot   normal   weak          No      NaN
```

```
[4]: # Encoding categorical variables
label_encoders = {}
for column in data.columns[1:]: # Ignore 'Day' as it's an identifier
    le = LabelEncoder()
    data[column] = le.fit_transform(data[column])
    label_encoders[column] = le
```

```
[5]: # Separating features and target variable
X = data[['Outlook', 'Temp', 'Humidity', 'Wind']] # Feature columns
y = data['PlayTennis'] # Target column
```

```
[6]: # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
```

```
[7]: # Train the Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
[8]: y_pred
```

```
[8]: array([1, 1, 0])
```

```
[9]: y_test
```

```
[9]: 9      1
    11     1
    0      0
    Name: PlayTennis, dtype: int64
```

```
[10]: # Calculate and print accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model accuracy:", accuracy)
```

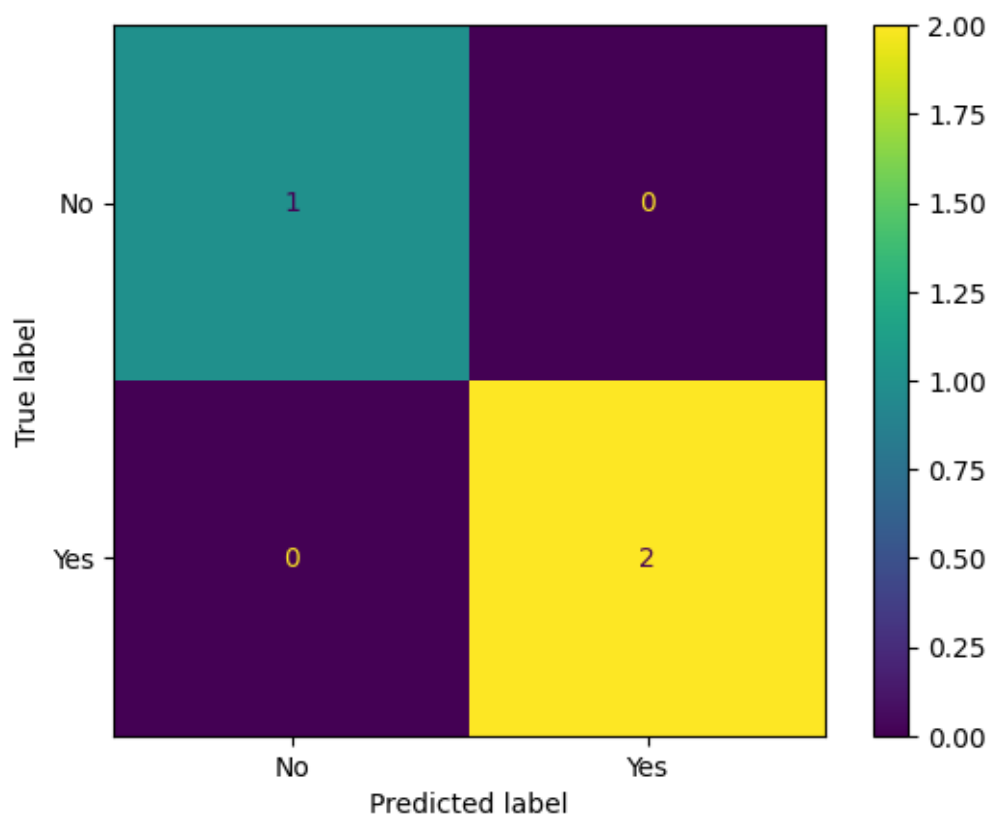
Model accuracy: 1.0

```
[11]: # Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
    ↳display_labels=label_encoders['PlayTennis'].classes_)
```

Confusion Matrix:

```
[[1 0]
 [0 2]]
```

```
[11]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7f2c94464e50>
```



```
[12]: # Show actual vs predicted values for each test sample
print("\nActual vs Predicted:")
for actual, predicted in zip(y_test, y_pred):
```

```
print(f"Actual: {label_encoders['PlayTennis'].
↪inverse_transform([actual])[0]}, {f"Predicted:
↪{label_encoders['PlayTennis'].inverse_transform([predicted])[0]}")
```

Actual vs Predicted:

Actual: Yes, Predicted: Yes

Actual: Yes, Predicted: Yes

Actual: No, Predicted: No

```
[13]: # Example prediction for new input {rain, mild, high, strong}
input_data = pd.DataFrame({
    'Outlook': [label_encoders['Outlook'].transform(['rain'])[0]],
    'Temp': [label_encoders['Temp'].transform(['mild'])[0]],
    'Humidity': [label_encoders['Humidity'].transform(['high'])[0]],
    'Wind': [label_encoders['Wind'].transform(['strong'])[0]]
})
```

```
[14]: # Predict whether to play tennis
prediction = model.predict(input_data)
result = label_encoders['PlayTennis'].inverse_transform(prediction)
print("\nPrediction for input {rain, mild, high, strong}: Play Tennis =",
↪result[0])
```

Prediction for input {rain, mild, high, strong}: Play Tennis = No

4 Decision tree from Dataset to predict as a Regressor

```
[15]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

```
[16]: # Load the dataset
data = pd.read_csv('house_data_DT.csv')
```

```
[17]: # Ensure all column names are stripped of whitespace
data.columns = data.columns.str.strip()
# Check if 'Location' exists
if 'Location' not in data.columns:
    raise KeyError("'Location' column is missing!")

# Convert all values in the 'Location' column to strings and strip whitespace
data['Location'] = data['Location'].astype(str).str.strip()

# Encode the 'Location' column
data['Location'] = data['Location'].map({'Suburb': 0, 'City': 1})

# Check for missing or unmapped values
if data['Location'].isnull().any():
```

```

print("Warning: Some rows in 'Location' have unmapped or invalid values!")
↪")
print(data[data['Location'].isnull()])
data = data.dropna() # Drop rows with unmapped values if necessary

```

```

[18]: # Define features (X) and target (y)
X = data[['Rooms', 'Area', 'Location']]
y = data['Price']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↪random_state=42)

```

```

[19]: # Train the Decision Tree Regressor
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X_train, y_train)

```

```

[19]: DecisionTreeRegressor(random_state=42)

```

```

[20]: # Make predictions on the test set
y_pred = regressor.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"R^2 Score: {r2}")

```

Mean Squared Error: 2500000000.0

R^2 Score: -10.111111111111111

```

[21]: # Function to predict price based on user input
def predict_price(rooms, area, location):
    # Encode the location input to match training data encoding
    location_encoded = 0 if location.lower() == 'suburb' else 1

    # Prepare the input data
    input_data = pd.DataFrame({
        'Rooms': [rooms],
        'Area': [area],
        'Location': [location_encoded]
    })

    # Make the prediction
    predicted_price = regressor.predict(input_data)
    print("\nPredicted Price for input (Rooms: {}, Area: {}, Location: {}):
↪${:,.2f}".format(rooms, area, location, predicted_price[0]))

# Example prediction
predict_price(rooms=3, area=75, location='Suburb')
predict_price(rooms=4, area=95, location='City')

```

Predicted Price for input (Rooms: 3, Area: 75, Location: Suburb): \$320,000.00

Predicted Price for input (Rooms: 4, Area: 95, Location: City): \$450,000.00

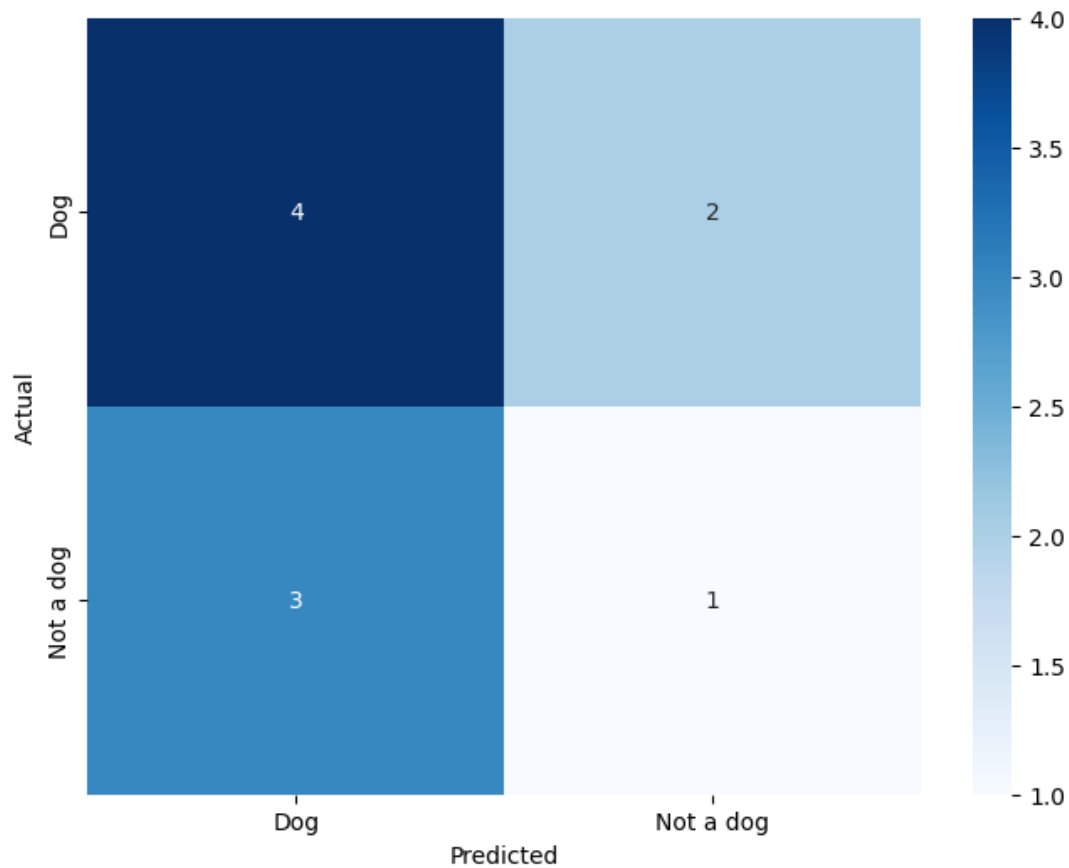
5 Confusion Matrix

```
[22]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix , classification_report
```

```
[23]: def plot_confusion_matrix(conf_matrix, class_names):
    df_cm = pd.DataFrame(conf_matrix, index=class_names, columns=class_names)
    plt.figure(figsize=(8, 6))
    sns.heatmap(df_cm, annot=True, fmt='d', cmap='Blues')
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()
```

```
[24]: truth = ["Dog","Not a dog","Dog","Dog", "Dog", "Not a dog", "Not a dog",
    ↪ "Dog", "Dog", "Not a dog"]
prediction = ["Dog","Dog", "Dog","Not a dog","Dog", "Not a dog", "Dog", "Not
    ↪ a dog", "Dog", "Dog"]
```

```
[25]: from sklearn.metrics import confusion_matrix
# Assuming 'truth' and 'prediction' are defined earlier in your code
cm = confusion_matrix(truth, prediction)
plot_confusion_matrix(cm, ["Dog", "Not a dog"])
```

```
[26]: print(classification_report(truth, prediction))
```

	precision	recall	f1-score	support
Dog	0.57	0.67	0.62	6
Not a dog	0.33	0.25	0.29	4
accuracy			0.50	10
macro avg	0.45	0.46	0.45	10
weighted avg	0.48	0.50	0.48	10

5.1 f1 score for “Dog” class

```
[27]: 2*(0.57*0.67/(0.57+0.67))
```

```
[27]: 0.6159677419354839
```

5.2 f1 score for “not a Dog” class

```
[28]: 2*(0.33*0.25/(0.33+0.25))
```

```
[28]: 0.2844827586206896
```